RAG

Manaranjan Pradhan

About Me



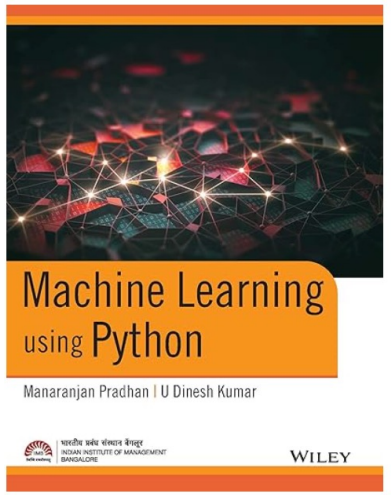
- Consulting and training on Big data, AI & Machine Learning.
- An alumni of *IIM, Bangalore*.
- Has about 20+ years of industry experience.
- Has trained 1000+ professionals on Big Data and AI & ML.
- An adjunct faculty at IIM, Bangalore, ISB, Hyderabad and Jio Institute

https://www.manaranjanp.com/

Manaranjan has co-authored the best-selling book Machine Learning using Python

He has published the following machine learning cases in **(HBR) Harvard Business Publishing**:

1. Customer Analytics at Big Basket – Product Recommendations
2. Improving Lead Generation at Eureka Forbes Using Machine Learning Algorithms



How do models understand language?

TF-IDF Representation

Sentences:

1. "the movie is just average."
2. "I loved the movie."
3. "what a terrible movie."

- Key Indicator Words: TF-IDF identifies the most important words
- Weighting: Words that are frequent in a particular document but rare across the corpus (like "delicious") will receive a higher weight, helping to focus on words that best capture the sentiment.

TF-IDF Representation

Sentences:

1. "the movie is just average."
2. "I loved the movie."
3. "what a terrible movie."

$$w_{x,y} = tf_{x,y} \times \log\left(\frac{N}{df_x}\right)$$

TF-IDF

Term x within document y

$tf_{x,y}$ = frequency of x in y

df_x = number of documents containing x

N = total number of documents

1. Dictionary Creation:

To create a dictionary of unique terms (words) in the corpus:

- Dictionary: ["the", "movie", "is", "just", "average", "I", "loved", "a", "terrible", "what"]

TF Representation

Sentence	the	movie	is	just	average	I	loved	what	a	terrible
the movie is just average	0.2	0.2	0.2	0.2	0.2	0.0	0.0	0.0	0.0	0.0
I loved the movie	0.25	0.25	0.0	0.0	0.0	0.25	0.25	0.0	0.0	0.0
what a terrible movie	0.0	0.25	0.0	0.0	0.0	0.0	0.0	0.25	0.25	0.25

IDF Calculation

Sentences:

1. "the movie is just average."
2. "I loved the movie."
3. "what a terrible movie."

$$IDF(t) = \log(N / (df_t))$$

Where N = Total number of sentences = 3

Term	Document Frequency (df_t)	IDF
the	2	log(3/2) = 0.4055
movie	3	log(3/3) = 0.0
is	1	log(3/1) = 1.0986
just	1	log(3/1) = 1.0986
average	1	log(3/1) = 1.0986
I	1	log(3/1) = 1.0986
loved	1	log(3/1) = 1.0986
what	1	log(3/1) = 1.0986
a	1	log(3/1) = 1.0986
terrible	1	log(3/1) = 1.0986

TF-IDF Representation

Sentence	the	movie	is	just	average	I	loved	what	a	terrible
the movie is just average	0.0811	0.0	0.2197	0.2197	0.2197	0.0	0.0	0.0	0.0	0.0
I loved the movie	0.1014	0.0	0.0	0.0	0.0	0.2746	0.2746	0.0	0.0	0.0
what a terrible movie	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.2746	0.2746	0.2746

TF-IDF Vector

```
tfidf_vectorizer = TfidfVectorizer(stop_words='english',
                                   min_df = 5,
                                   max_df = 0.6)
```

Stop words in Natural Language Processing (NLP) refer to the most common words in a language that hold little value in text analysis, as they don't carry significant meaning by themselves.

Removing stop words can sometimes lead to loss of context, especially in cases where the stop word has a specific meaning in the context of the text. Therefore, the use or exclusion of stop words should be decided based on the specific use case and the context of the text.

Stop Words

These words include:

- a
- I
- the
- in
- of
- for
- at
- to
- on
- with
- from

https://botpenguin.com/glossary/stop-words

Sparse Representation

communicate	blueprint	types	discipline	view	setting	useful	energy	help	significant	know	risk	perspective	identifies	presents	young
0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.0	0.000000
0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.0	0.000000
0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.0	0.000000
0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.0	0.000000
0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.404743	0.0	0.0	0.0	0.0	0.000000
0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.0	0.223512
0.0	0.0	0.0	0.0	0.0	0.0	0.333147	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.0	0.231962
0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.0	0.000000
0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.0	0.000000
0.0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.0	0.000000

Limitations of TF-IDF

1. Lack of Semantic Understanding.
2. Sparse Vectors: dealing with a large vocabulary and a collection of documents.
3. Loss of Word Order: Does not preserve word order or context within a document. It treats documents as bags of words
4. Sensitivity to Term Variations: TF-IDF may treat variations of the same term (e.g., "cat" and "cats") as separate terms.
5. Difficulty Handling Synonyms and Polysemy: TF-IDF struggles with synonyms (words with similar meanings) and polysemy (words with multiple meanings).
6. Scaling Issues: computationally expensive as the corpus size increases.

Embeddings

How do we represent words or sentences or text in the world of numbers?

What is an Embedding?

Roles	Leadership	Technical Expertise	Financial Acumen	Operational Efficiency	Talent Acquisition	Strategic Planning	Innovation
CEO	0.9	0.6	0.8	0.7	0.6	0.9	0.8
CFO	0.7	0.5	0.95	0.7	0.5	0.8	0.6
CTO	0.7	0.95	0.6	0.7	0.5	0.8	0.9
COO	0.8	0.6	0.7	0.95	0.6	0.8	0.7
CHRO	0.8	0.5	0.6	0.7	0.95	0.7	0.6

Disclaimer: This is only for demonstration purpose. May not reflect the actual expertise in each area.

CEO [0.9, 0.6, 0.8, 0.7, 0.6, 0.9, 0.8]

So, the role of a CEO can be represented **a series of numbers** in vector space and it is called embedding representation on the dimension of expertise they posses.

Embeddings: Foundation of GenAI

But how to define dimensions (features or factors) for representing semantics of millions of possible words (vocabulary).

Representing with Latent Factors

man →	0.6	−0.2	0.8	0.9	−0.1	−0.9	−0.7
woman →	0.7	0.3	0.9	−0.7	0.1	−0.5	−0.4
king →	0.5	−0.4	0.7	0.8	0.9	−0.7	−0.6
queen →	0.8	−0.1	0.8	−0.9	0.8	−0.5	−0.9

But how do we know this is correct representation?

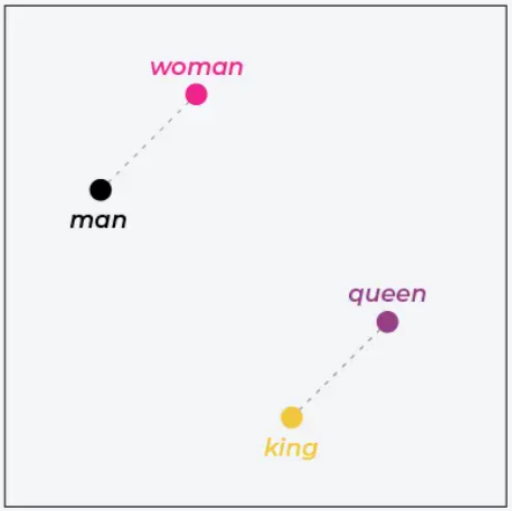
The embeddings have semantics?

Efficient Estimation of Word Representations in Vector Space

Tomas Mikolov, Kai Chen, Greg Corrado, Jeffrey Dean

```
king - man + woman ~= queen
```

Paris - France + India ~= New Delhi



https://arxiv.org/pdf/1301.3781.pdf

GloVe: Global Vectors for Word Representation

Created from **Common Crawl** using Self Supervised Learning

- 840B tokens,
- 2.2M vocab,
- 300d vectors

glove.840B.300d.zip

Using Self Supervised Techniques

https://nlp.stanford.edu/projects/glove/

Word2Vec

- First billion characters from wikipedia (use the pre-processing perl script from the bottom of Matt Mahoney's page)
- Latest Wikipedia dump Use the same script as above to obtain clean text. Should be more than 3 billion words.
- WMT11 site: text data for several languages (duplicate sentences should be removed before training the models)
- Dataset from "One Billion Word Language Modeling Benchmark" Almost 1B words, already pre-processed text.
- UMBC webbase corpus Around 3 billion words, more info here. Needs further processing (mainly tokenization).
- Text data from more languages can be obtained at statmt.org and in the Polyglot project.



word2vec

https://code.google.com/archive/p/word2vec/

fastText



Pre-trained word vectors learned on different sources can be downloaded below:

1. wiki-news-300d-1M.vec.zip: 1 million word vectors trained on Wikipedia 2017, UMBC webbase corpus and statmt.org news dataset (16B tokens).
2. wiki-news-300d-1M-subword.vec.zip: 1 million word vectors trained with subword infomation on Wikipedia 2017, UMBC webbase corpus and statmt.org news dataset (16B tokens).
3. crawl-300d-2M.vec.zip: 2 million word vectors trained on Common Crawl (600B tokens).
4. crawl-300d-2M-subword.zip: 2 million word vectors trained with subword information on Common Crawl (600B tokens).

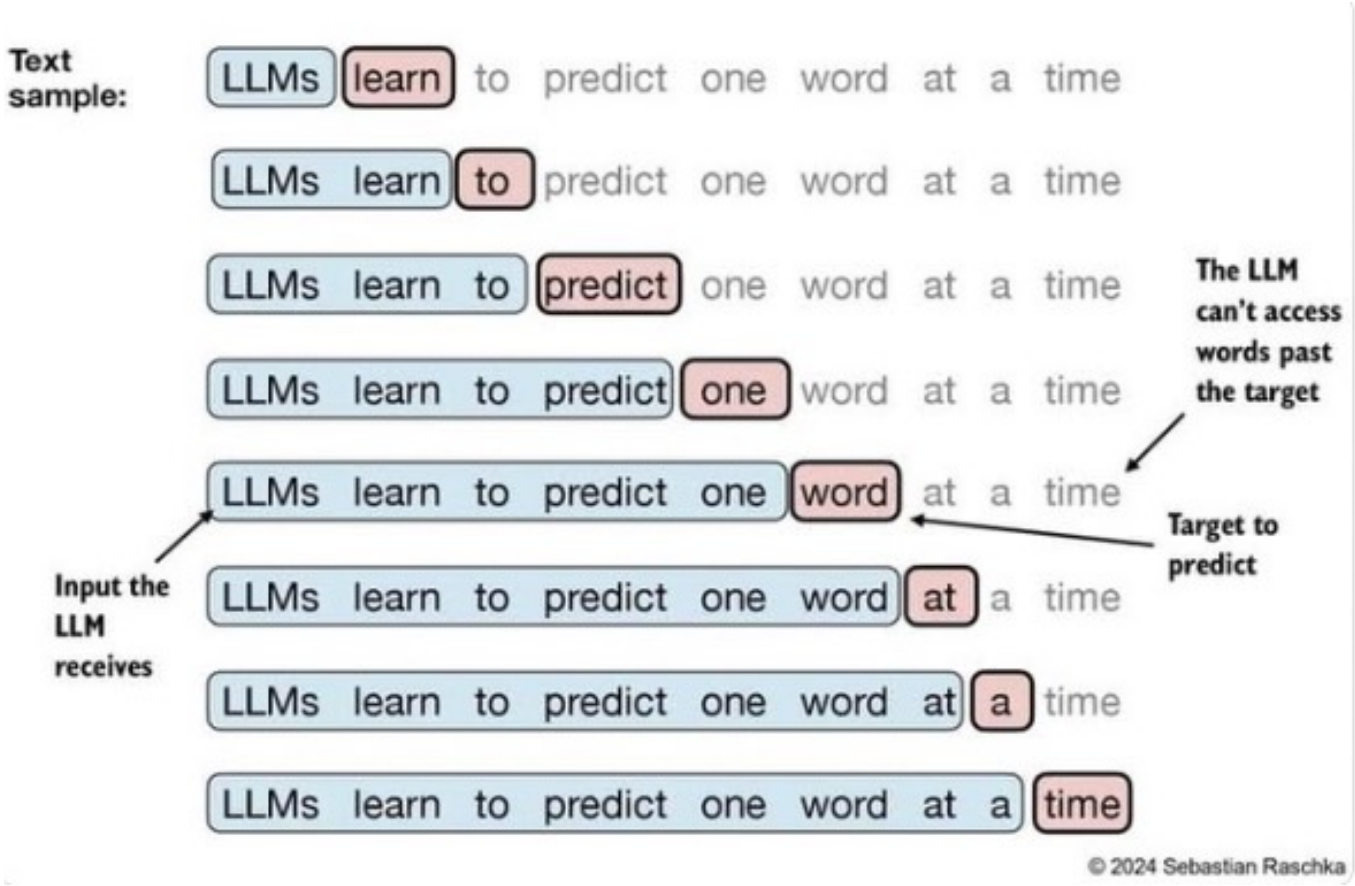
Word vectors for 157 languages

We distribute pre-trained word vectors for 157 languages, trained on *Common Crawl* and *Wikipedia* using fastText. These models were trained using CBOW with position-weights, in dimension 300, with character n-grams of length 5, a window of size 5 and 10 negatives. We also distribute three new word analogy datasets, for French, Hindi and Polish.

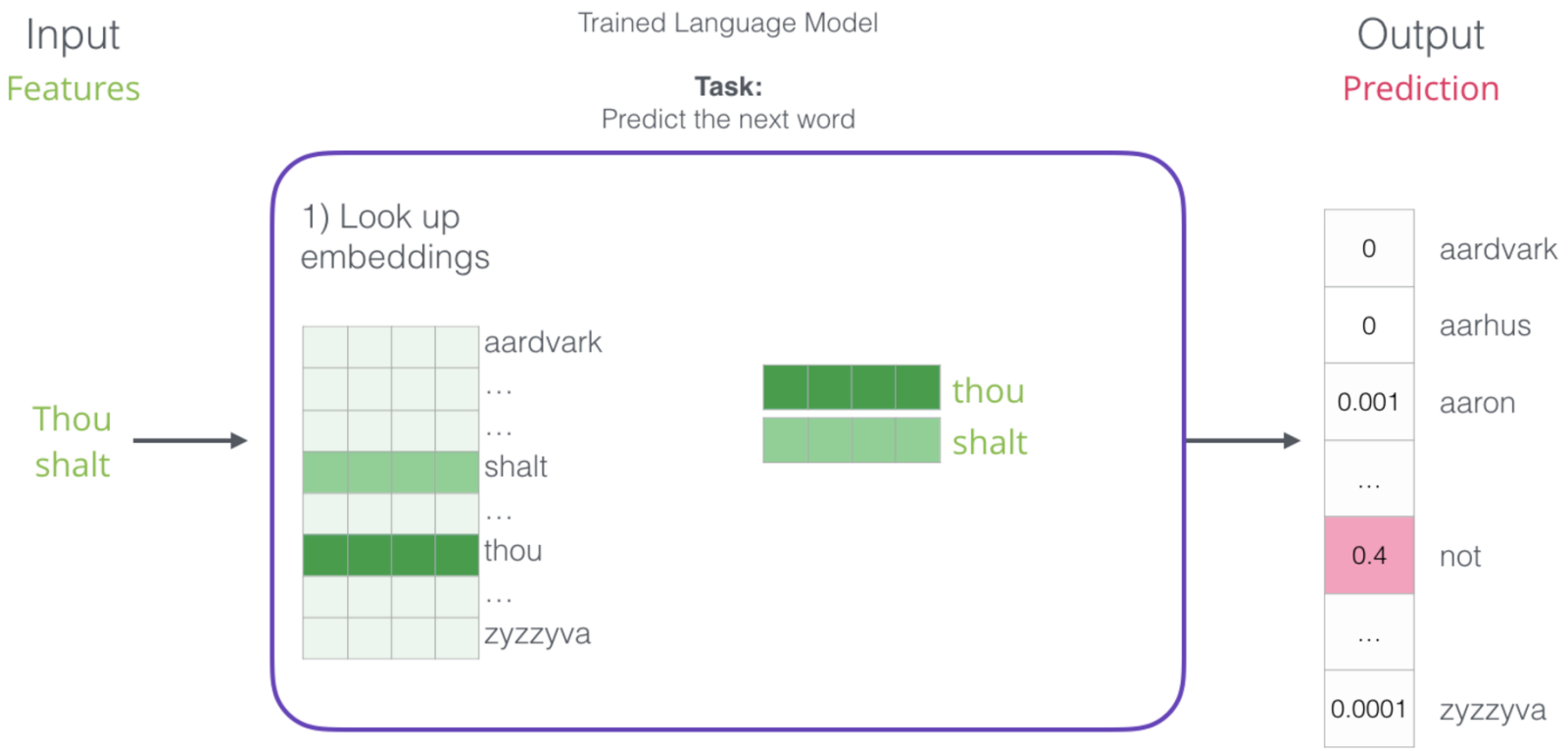
https://fasttext.cc/docs/en/crawl-vectors.html

‘

Next Word Prediction



Learn to represent words in vectors



Embedding Leaderboard

1	gemini-embedding-exp-03-07	99%	79.28	**71.82**	**54.99**	5.18	**29.16**
2	Linq-Embed-Mistral	99%	70.34	62.24	51.27	0.94	24.77
3	gte-Qwen2-7B-instruct	⚠ NA	73.92	61.55	53.36	4.94	25.48
4	multilingual-e5-large-instruct	99%	**80.13**	64.94	51.54	-0.40	22.91
5	SFR-Embedding-Mistral	96%	70.00	60.02	52.57	0.16	24.55
6	GritLM-7B	99%	70.53	61.83	50.48	3.45	22.77
7	text-multilingual-embedding-002	99%	70.73	64.64	48.47	4.08	22.80
8	GritLM-8x7B	99%	68.17	61.55	50.88	2.44	24.43
9	e5-mistral-7b-instruct	99%	70.58	60.31	51.39	-0.62	22.20
10	Cohere-embed-multilingual-v3.0	⚠ NA	70.50	62.95	47.61	-1.89	22.74
11	gte-Qwen2-1.5B-instruct	⚠ NA	62.51	58.32	52.59	0.74	24.02

Manaranjan Pradhan (manaranjan@gmail.com)

Demo - Embeddings

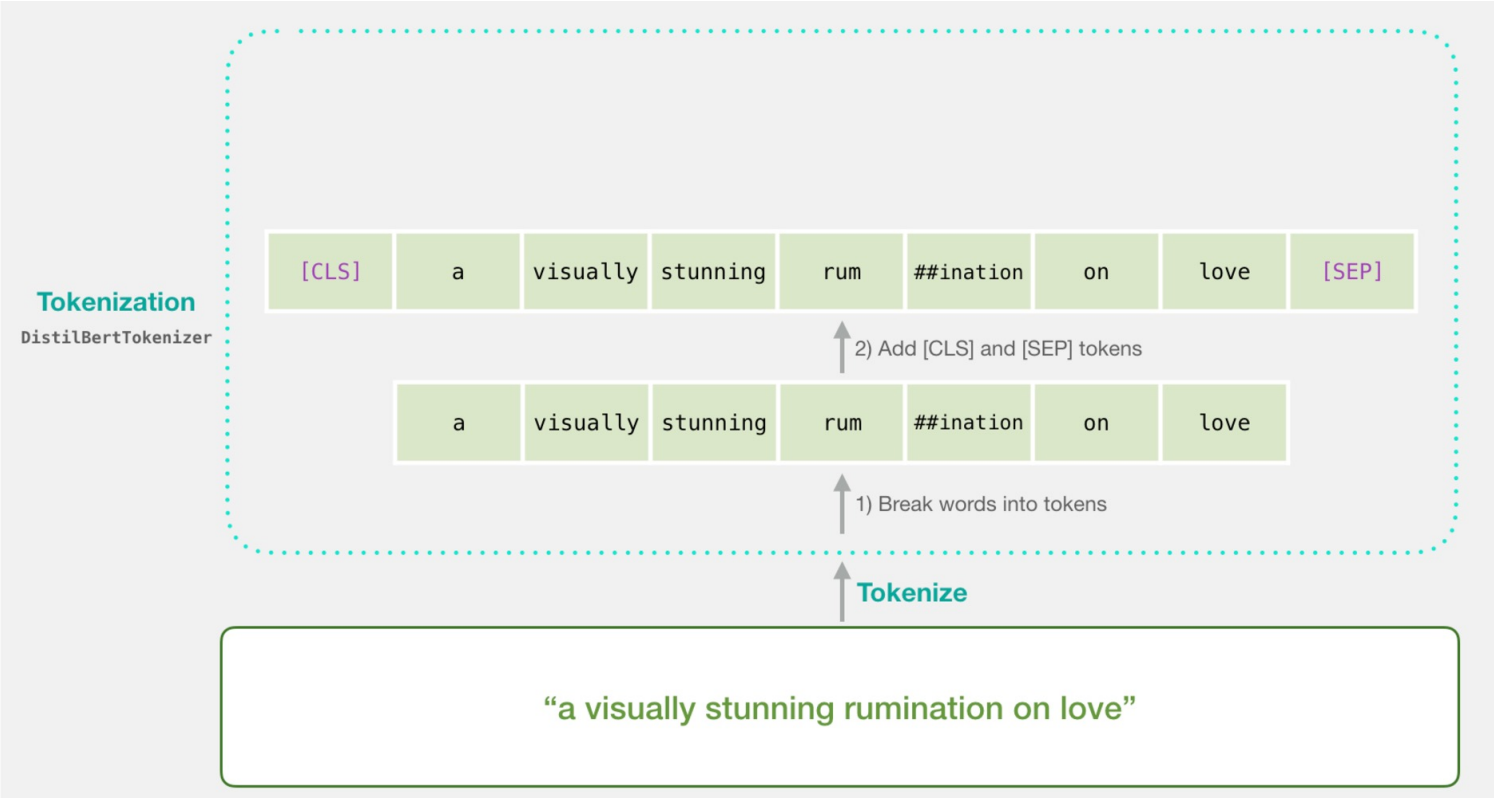
But there are too many words to learn!

Tokenization



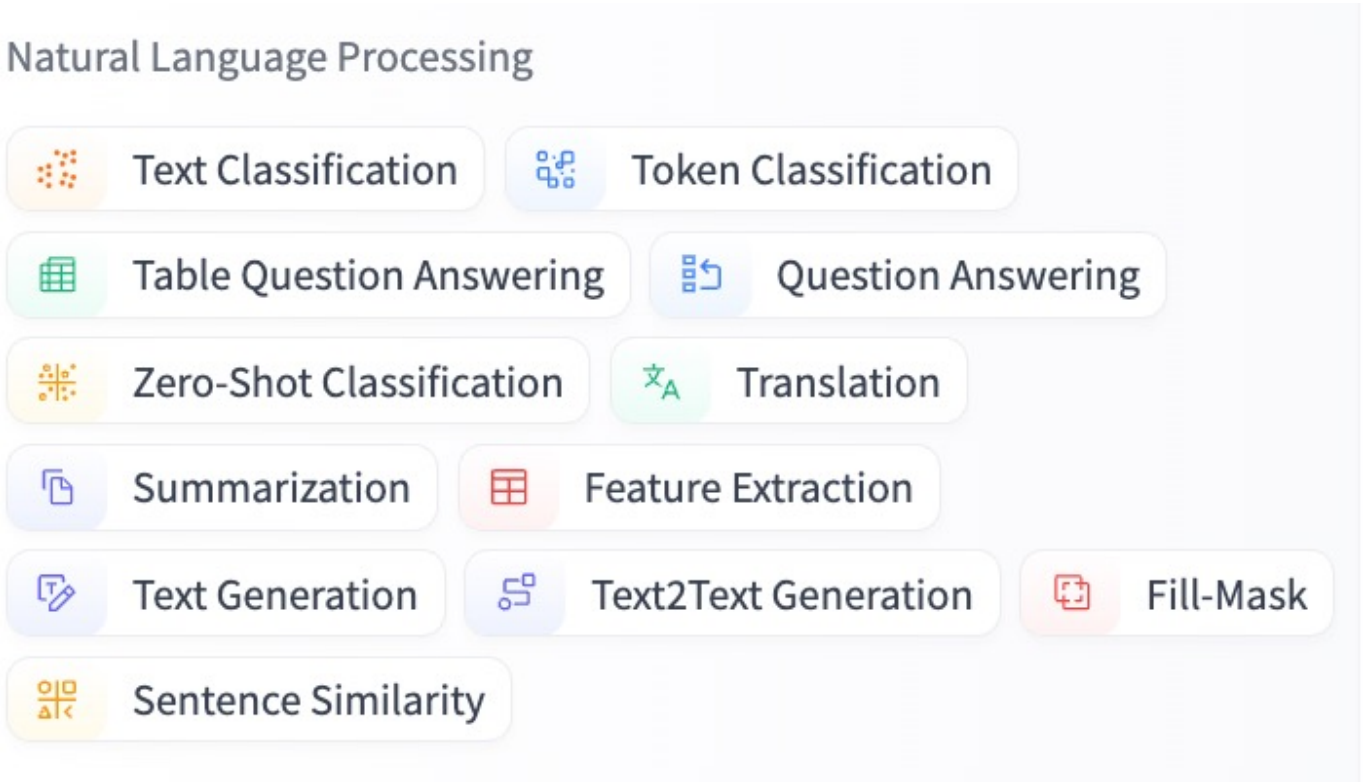
https://huggingface.co/spaces/Xenova/the-tokenizer-playground
https://www.thoughtvector.io/blog/subword-tokenization/

Tokens



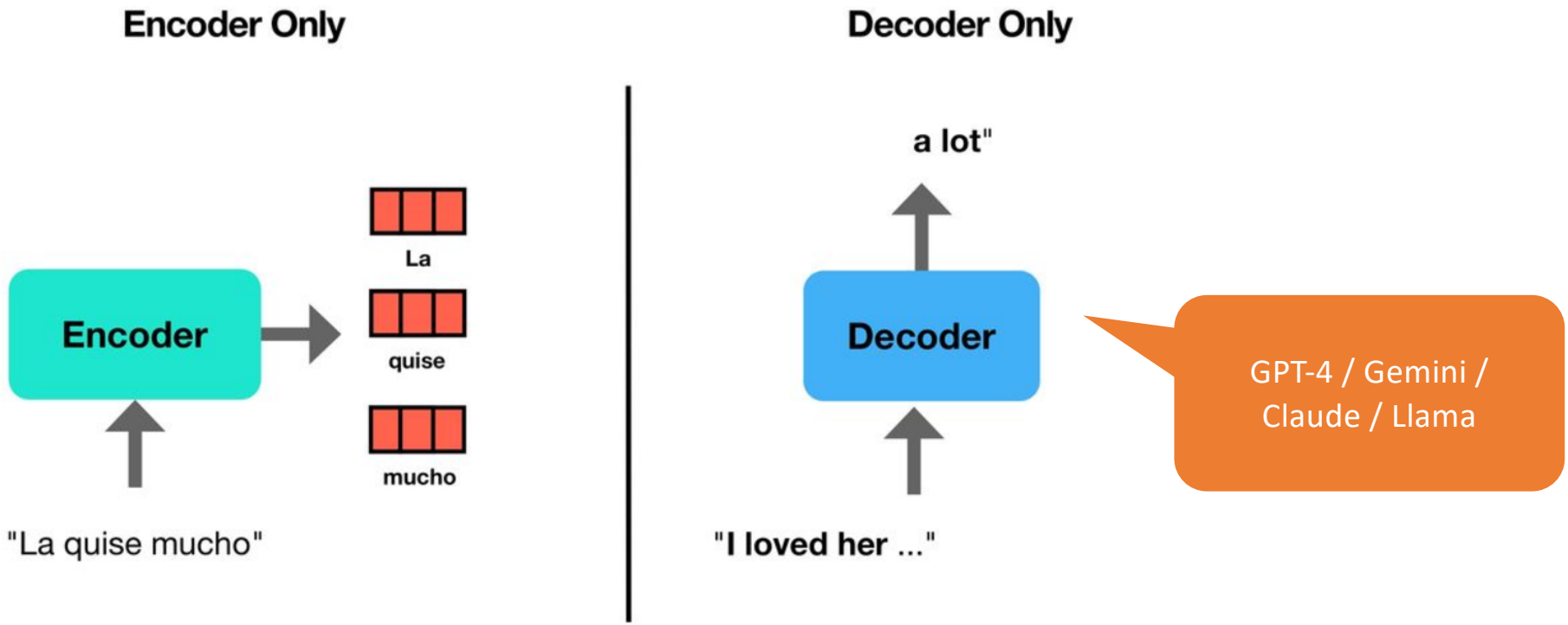
Subword tokenization breaks words into smaller parts, like "play" and "ing" in "playing." This helps models understand and work with rare or new words, reduces the number of words they need to learn, and makes them smarter at handling different languages.

Specialized Models to Generalized Models

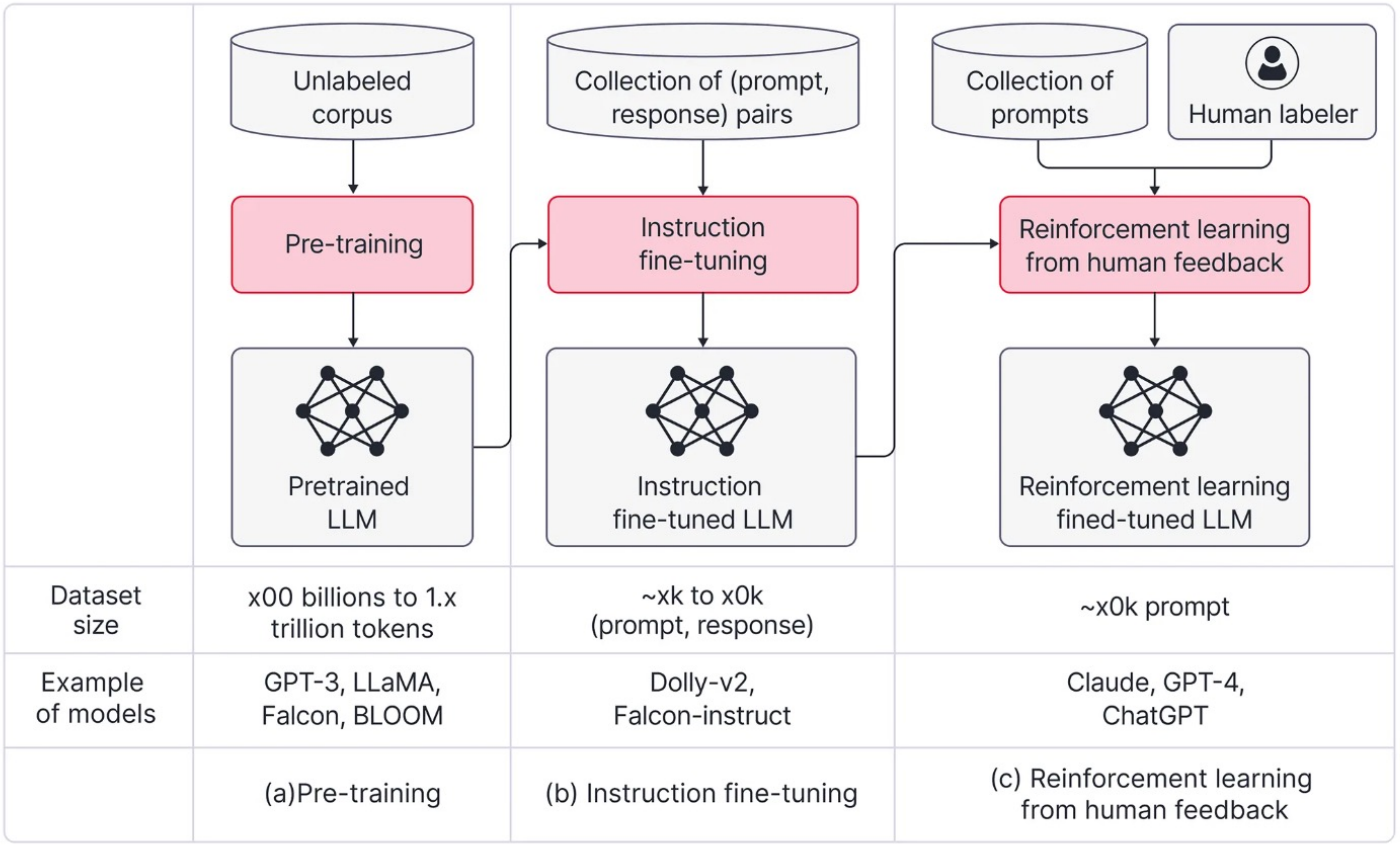


https://huggingface.co/models

Generative Models (Decoder Only)



How LLMs are built?



https://rbcborealis.com/research-blogs/a-high-level-overview-of-large-language-models/

Fineweb Dataset

Key Highlight	Description
Extensive Scale	Comprises over 15 trillion tokens of cleaned and deduplicated English web data, offering a vast resource for comprehensive language model training.
Diverse Data Sources	Aggregated from 96 Common Crawl snapshots spanning 2013 to 2024, encompassing a wide array of web pages, including news articles, blogs, and educational content.
Optimized Data Processing	Employs advanced filtering and deduplication techniques using the datatrove library to enhance data quality, crucial for effective LLM performance.
Open Access and Transparency	Released under the Open Data Commons Attribution License (ODC-By) v1.0, promoting transparency and collaboration within the AI community.
Superior Model Performance	Models trained on FineWeb have demonstrated improved performance compared to those trained on other high-quality web datasets across various benchmark tasks.

Data Size is always measured in terms of number of tokens

https://huggingface.co/spaces/HuggingFaceFW/blogpost-fineweb-v1

Pretraining

Step 1: Pre-training (Reading a Chapter)

Imagine the model "reads" the following paragraph during pre-training:

> "A stock represents ownership in a company and is traded on stock exchanges. The price of a stock is influenced by factors such as company performance, market trends, and economic conditions. Bonds, on the other hand, are debt instruments where investors lend money to issuers in exchange for periodic interest payments and the return of principal at maturity. Diversification is a key strategy to reduce risk in an investment portfolio."

Finetuning for multiple tasks

Fine-tuning for Financial Q&A:

- The model practices answering finance-related questions.
- Example exercise:
  - **Question:** "What is the primary purpose of diversification in investing?"
  - **Answer:** "To reduce risk in an investment portfolio."

Prompt-response pairs (instruction-output)

Fine-tuning for Sentiment Analysis:

- The model learns to analyze market sentiment based on news or reports.
- Example exercise:
  - **Task:** Analyze the sentiment in: "The company's quarterly earnings surpassed expectations, driving stock prices up."
  - **Answer:** Positive sentiment.

Fine-tuning for Text Summarization:

- The model summarizes financial articles or reports.
- Example exercise:
  - **Task:** Summarize the paragraph.
  - **Answer:** "Stocks offer ownership in companies, bonds provide fixed income, and diversification reduces risk."

Pretraining vs Finetuning

- Pre-training (Reading a Chapter):
  - Purpose: Acquire broad, general knowledge.
  - Process: Engage with extensive, diverse data to learn foundational patterns and structures.
  - Outcome: Develop a comprehensive understanding applicable to various contexts.
- Fine-tuning (Doing Exercises):
  - Purpose: Apply and refine knowledge for specific tasks.
  - Process: Focus on targeted, task-specific data to adjust and optimize performance.
  - Outcome: Enhance proficiency in particular areas, improving accuracy and relevance.

Instruction datasets

Dataset Name	Size	Language	Description	Source
Dolly 2.0 Dataset	15,000 prompts	English	A human-generated instruction dataset designed for training instruction-following LLMs.	Databricks Blog
Alpaca Dataset	52,000 prompts	English	Generated using OpenAI's text-davinci-003; tailored for instruction-following training.	Stanford CRFM
FLAN Collection	1.8M examples	Multilingual	A diverse set of tasks and instructions aimed at enhancing model generalization across tasks.	Google Research
OpenOrca Dataset	3.3M examples	English	Combines multiple datasets to improve reasoning capabilities and instruction-following in LLMs.	OpenOrca

Domain Specific Models

Model Name	Domain	Description	Reference
BloombergGPT	Finance	A large language model developed by Bloomberg, trained on a vast dataset of financial data to enhance tasks such as sentiment analysis, news classification, and financial question answering. It combines public datasets with proprietary financial data to achieve superior performance in financial applications.	Building Domain-Specific LLMs: Examples and Techniques
Med-PaLM 2	Medical	Developed by Google, this model is fine-tuned on curated medical datasets to assist in medical question answering and healthcare applications, demonstrating the potential of domain-specific models in specialized fields.	Building Domain-Specific LLMs: Examples and Techniques
Harvey	Legal	A platform trained on legal data to assist with tasks such as contract analysis and legal research. PwC has partnered with OpenAI and Harvey to develop and deploy foundation models tailored for tax, legal, and human resources services.	PwC partners with OpenAI and Harvey to build domain specific foundation models

Which is a better model?

Total #models: **169**. Total #votes: **2,296,813**. Last updated: 2024-11-21.

Code to recreate leaderboard tables and plots in this notebook. You can contribute your vote at lmarena.ai!

Category: Overall

Apply filter: Style Control, Show Deprecated

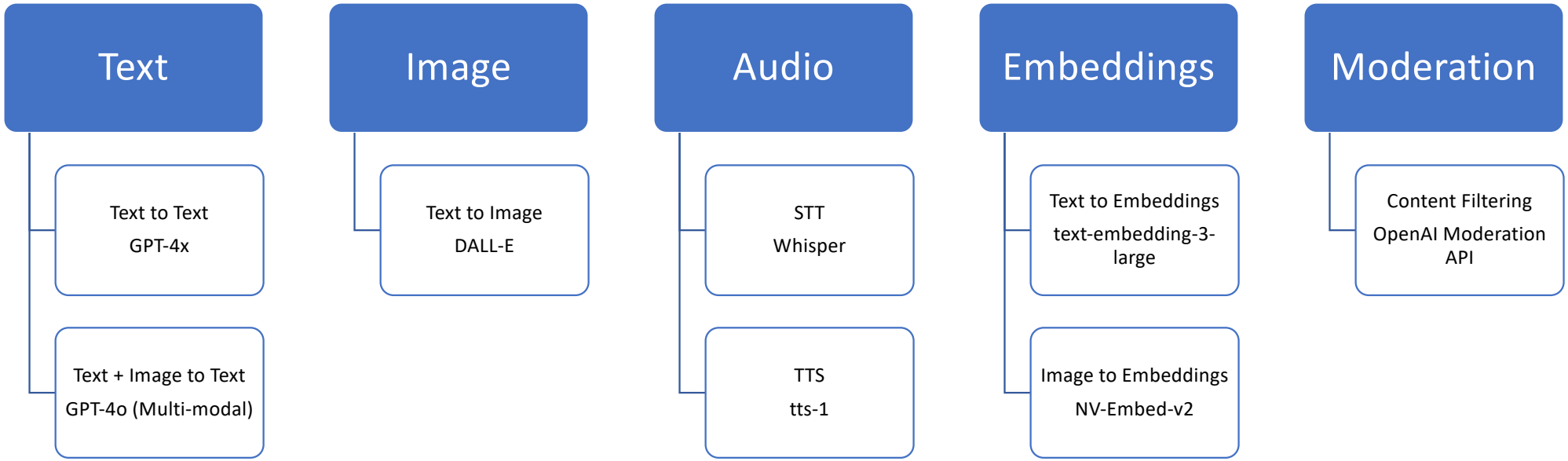
Overall Questions

#models: 169 (100%) **#votes: 2,296,813 (100%)**

Rank* (UB)	Rank (StyleCtrl)	Model	Arena Score	95% CI	Votes	Organization	License
1	2	Gemini-Exp-1121	1365	+8/-6	5625	Google	Proprietary
1	1	ChatGPT-4o-latest (2024-11-20)	1361	+4/-5	10658	OpenAI	Proprietary
3	5	Gemini-Exp-1114	1344	+4/-5	12778	Google	Proprietary
4	2	o1-preview	1334	+4/-4	27835	OpenAI	Proprietary
5	7	o1-mini	1308	+3/-4	31992	OpenAI	Proprietary
5	5	Gemini-1.5-Pro-002	1301	+5/-3	27336	Google	Proprietary
7	12	Yi-Lightning	1287	+4/-3	29336	01 AI	Proprietary
7	10	Grok-2-08-13	1289	+4/-3	52102	xAI	Proprietary
7	5	GPT-4o-2024-05-13	1285	+2/-2	111745	OpenAI	Proprietary

https://lmarena.ai/?leaderboard

Types of Models by Input/Output Types

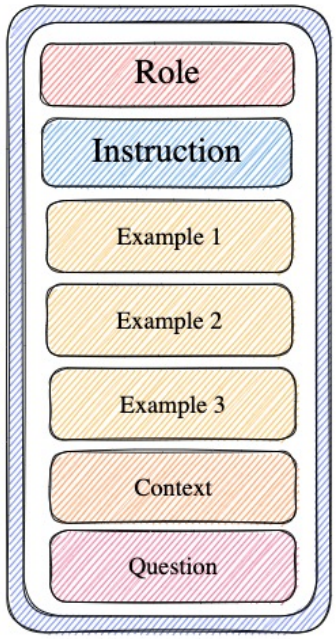


What is Prompting?

The act of providing a question or statement to elicit a desired response is often referred to as "prompting."

The art and science of asking a question to an LLM is called "prompt engineering"

Prompt Structure



- **Role:** Defines the identity or persona to guide model response.
- **Instruction:** Clearly specifies the task(s) to be performed.
- **Context:** Provides background information to clarify the task or additional information.
- **Example:** Example of completed tasks as references or learn from.
- **Question:** This is the main query or problem at hand to accomplish or complete.
- **Output Indicator:** Defines the desired format or structure of the response.

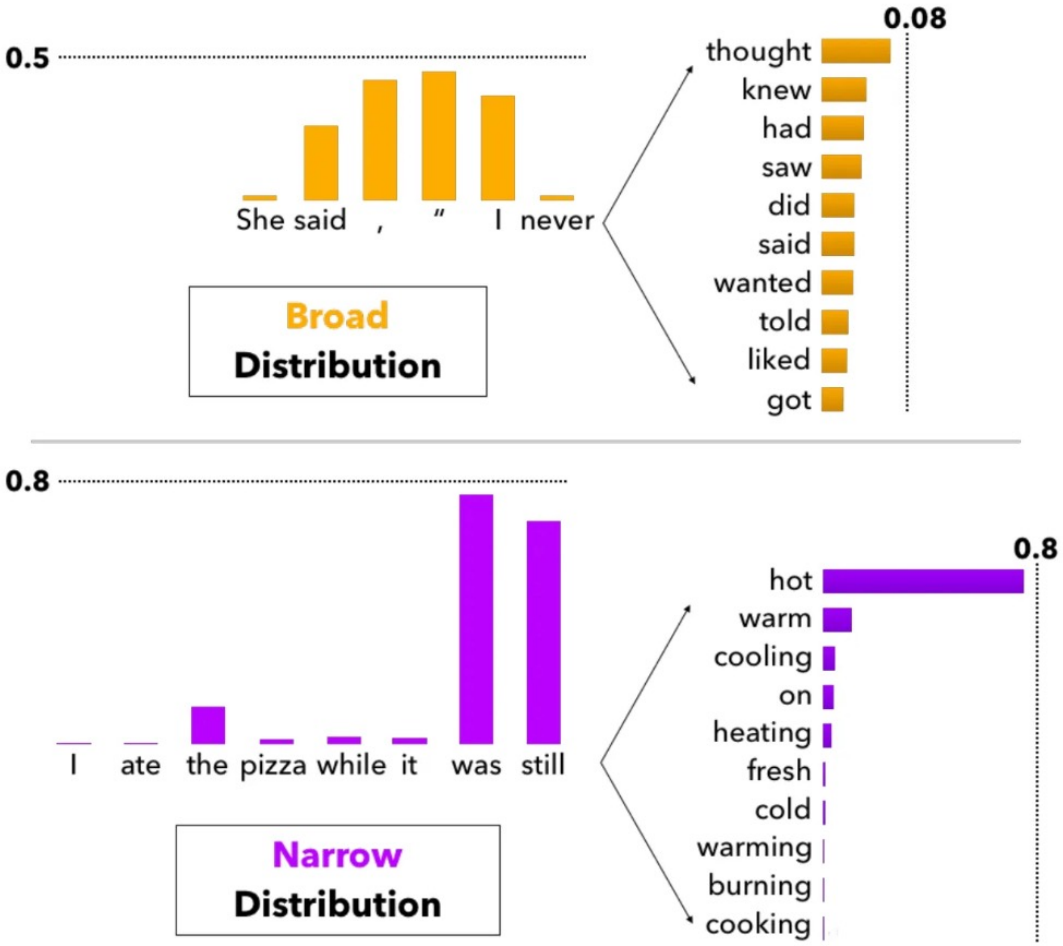
https://learnprompting.org/docs/basics/prompt_structure

Model Parameters to control generation

Parameter	Definition	How It Works	Effects	Typical Use Cases
Temperature	Controls randomness by scaling the logits before sampling.	- Adjusts probabilities of words. - Higher values explore less probable words; lower values focus on the most probable ones.	- **High values (e.g., 1.0+)**: Diverse, creative output. - **Low values (e.g., 0.2)**: Predictable, deterministic output.	- Creative writing (high). - Factual Q&A or technical tasks (low).
Top-p Sampling	Samples from a subset of words whose cumulative probability exceeds a threshold (p).	- Computes cumulative probabilities of words. - Selects a subset of the top probable words. - Normalizes and samples from the subset.	- **High (p = 0.9):** Diverse and flexible output. - **Low (p = 0.3):** More focused and constrained responses.	- Creative or open-ended tasks (high). - Focused tasks like summarization (low).

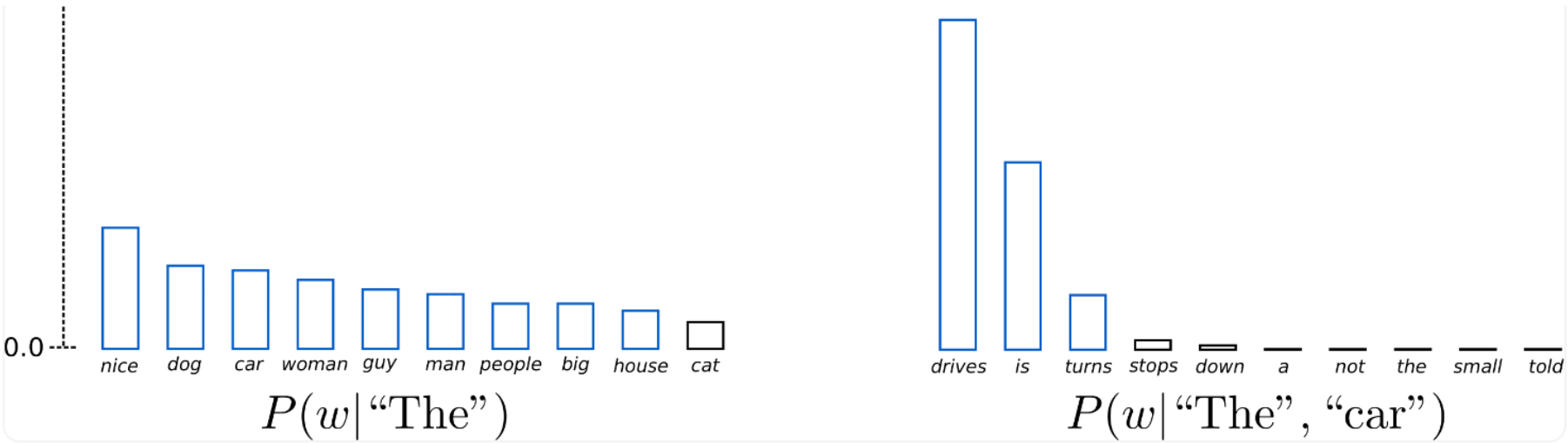
www.manaranjanp.com (manaranjan@gmail.com)

Temperature Sampling



Holtzman et al 2019

Top-P Sampling

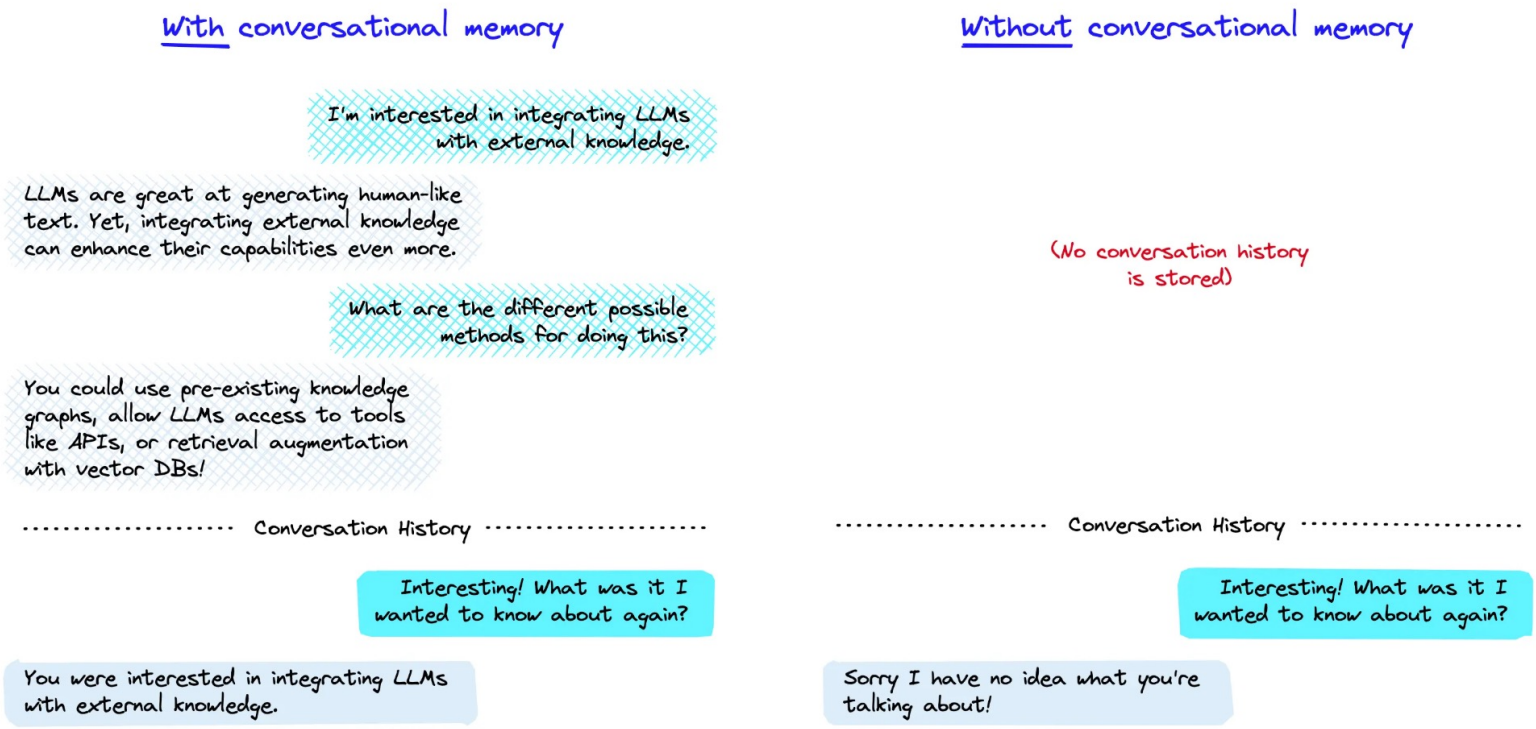


Top-p sampling chooses from the smallest possible set of words whose cumulative probability exceeds the probability *p*

https://huggingface.co/blog/how-to-generate

Does the model remember past interactions?

Memory In Conversation AI



https://www.pinecone.io/learn/series/langchain/langchain-conversational-memory/

Single Turn Vs Multi Turn Conversations

Aspect	Single-Turn Conversations	Multi-Turn Conversations
Context Retention	No context is retained between interactions.	Context is retained across multiple turns.
Complexity	Simple, one-step interactions.	Handles complex tasks requiring follow-up questions.
Use Case	Suitable for one-off queries like checking store hours.	Suitable for resolving issues like troubleshooting.
Technical Handling	Each query is treated as independent.	Conversation history must be provided with each input.
Example	User: *"What are your store hours today?"* Assistant: *"Our store is open from 9 AM to 8 PM."*	User: *"I have a problem with my order."* Assistant: *"Can you provide your order number?"* User: *"It's 12345."* Assistant: *"Your order is being processed and will arrive tomorrow."*

System, User, Assistant Messages

Message Type	Purpose	Who Provides It?	Example	Effect on Conversation
System Message	Provides context, guidelines, or instructions to set the tone, behavior, and purpose of the model.	Defined by the developer or operator.	*"You are a helpful assistant skilled in explaining programming concepts in simple terms."*	Shapes the assistant's behavior, tone, and focus for the entire interaction.
User Message	Contains the prompt or query from the user, asking for information or initiating a task.	Provided by the user.	*"What is the difference between supervised and unsupervised learning?"*	Directs the conversation by specifying what the assistant should respond to or focus on.
Assistant Message	Contains the response generated by the LLM, addressing the user query or following the system's rules.	Generated by the LLM.	*"Supervised learning involves labeled data, while unsupervised learning involves finding patterns in unlabeled data."*	Provides the answer or output based on the user's query and the system message context.

Single Turn Conversations: API

```python
import openai

# OpenAI API key setup
openai.api_key = "your-api-key"

# Single-turn conversation
response = openai.ChatCompletion.create(
    model="gpt-4",
    messages=[
        {"role": "system", "content": "You are a helpful customer support assistan
        {"role": "user", "content": "What are your store hours today?"}
    ]
)

# Print the assistant's response
print(response['choices'][0]['message']['content'])
```

Multi Turn Conversations

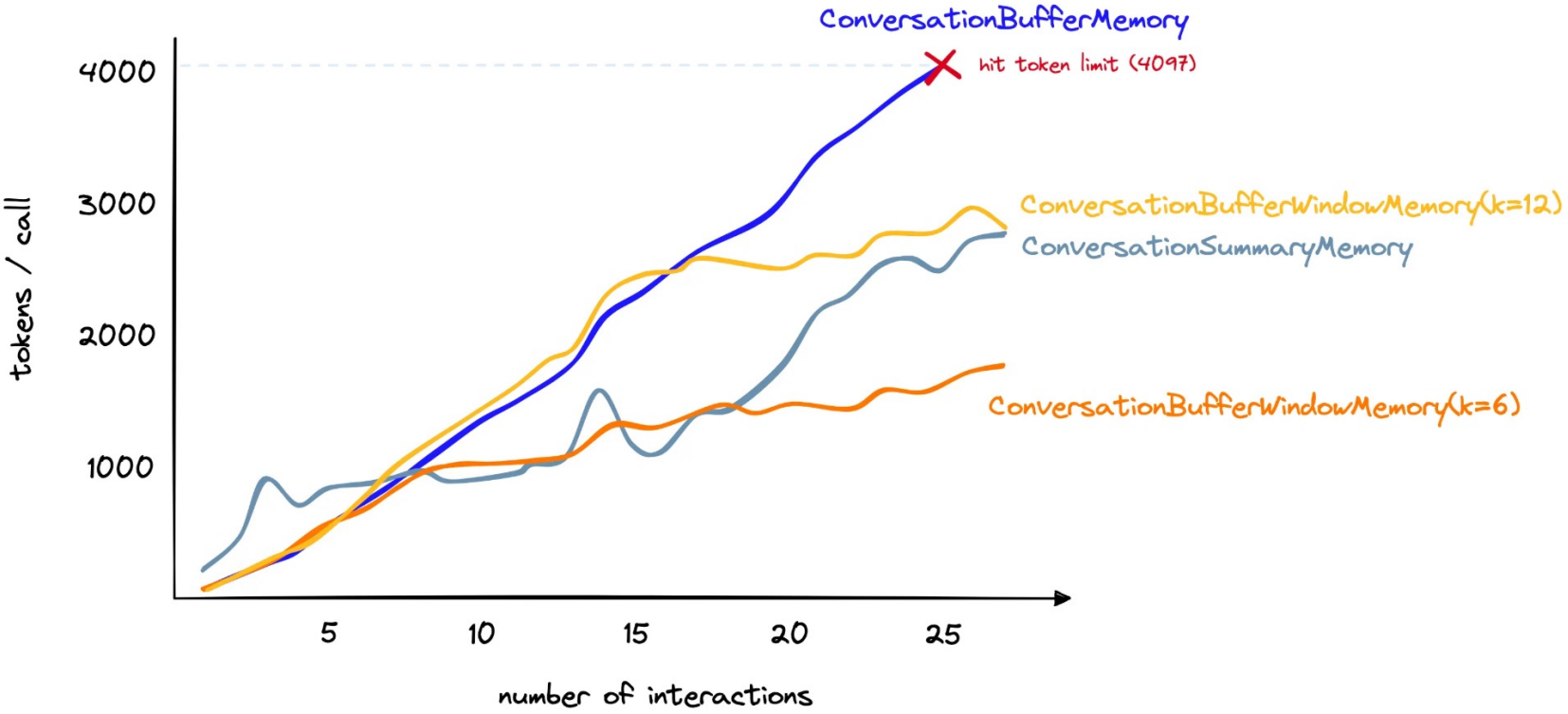
```
# OpenAI API key setup
openai.api_key = "your-api-key"

# Initial conversation history with 3 past interactions
messages = [
    {"role": "system", "content": "You are a helpful customer support assistant."}
    {"role": "user", "content": "I need help with my order."},
    {"role": "assistant", "content": "Sure, I can help. Can you please provide you
    {"role": "user", "content": "The order number is 12345."},
    {"role": "assistant", "content": "Thank you. I see that your order is being pr
]

# User's new message
messages.append({"role": "user", "content": "Can you change the delivery address?"

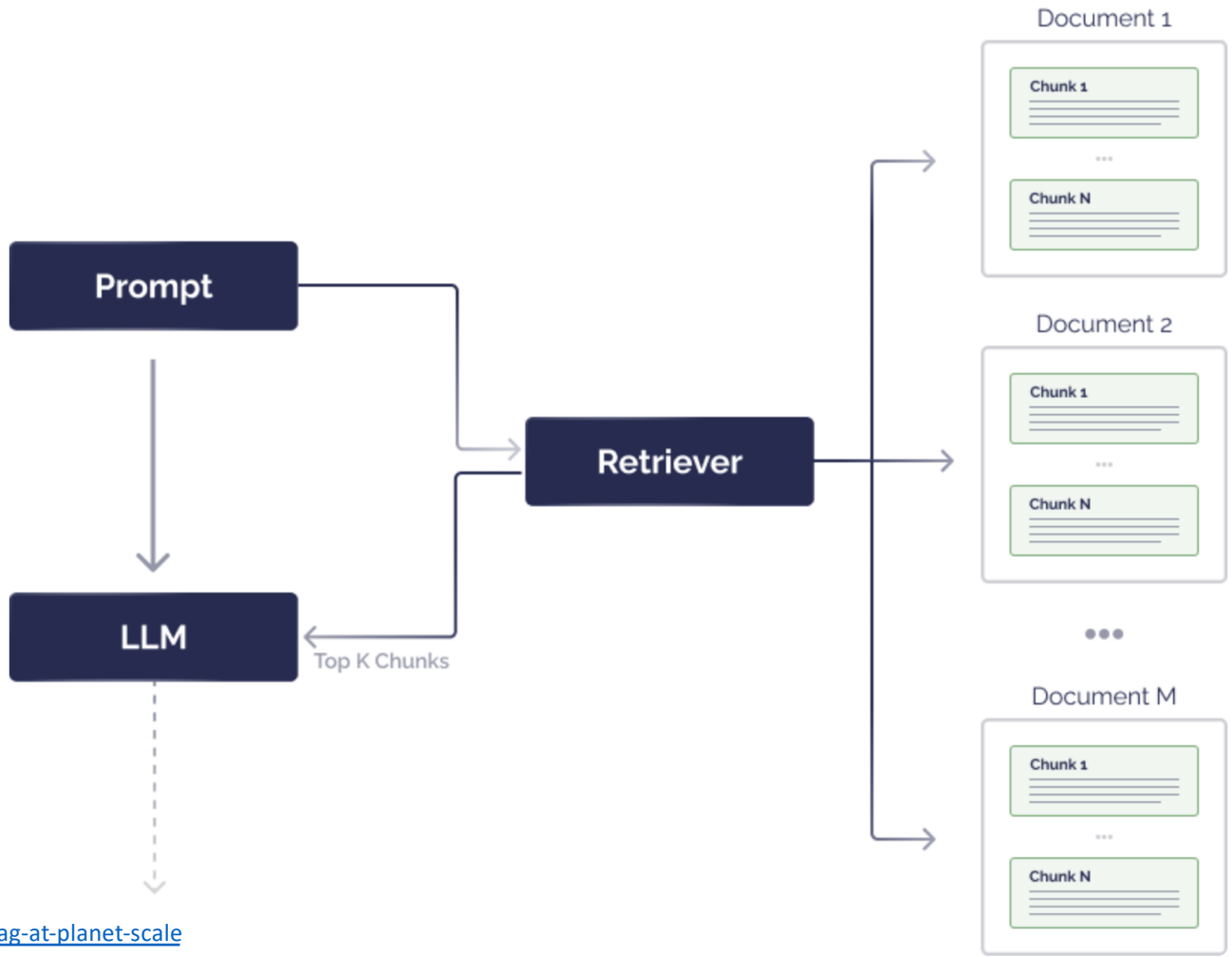
# Make API call
response = openai.ChatCompletion.create(
    model="gpt-4",
    messages=messages
)
```

Conversational Memory



https://www.pinecone.io/learn/series/langchain/langchain-conversational-memory/

RAG

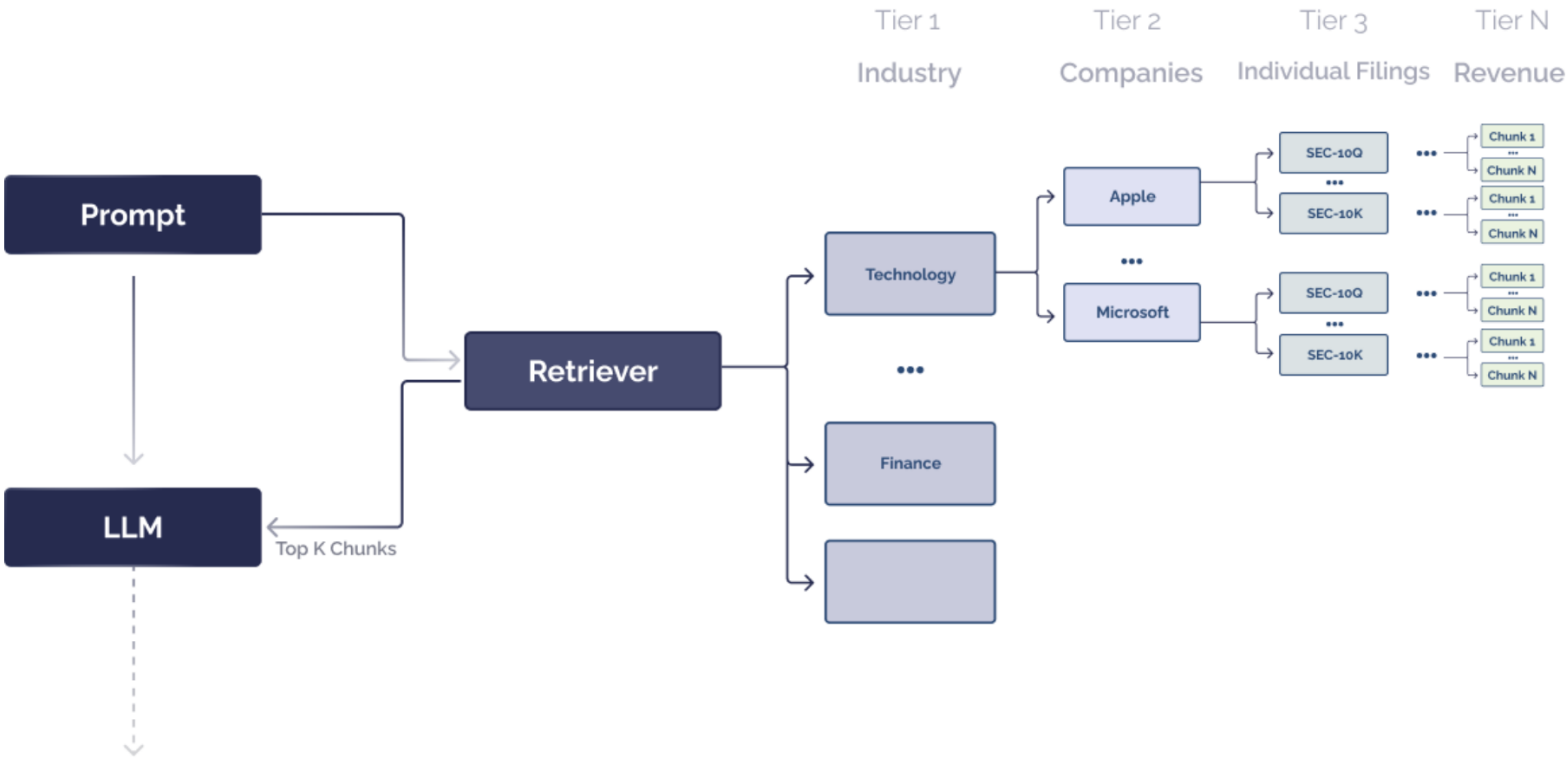


https://www.arcus.co/blog/rag-at-planet-scale

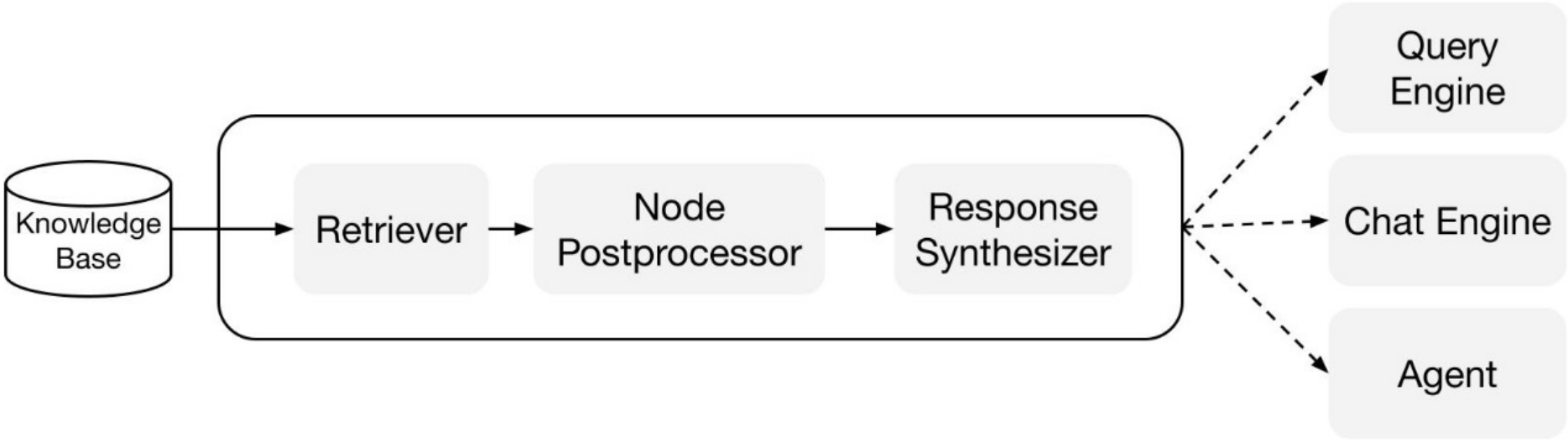
Stage 1: Indexing Stage



Multi-tier approach of Indexing



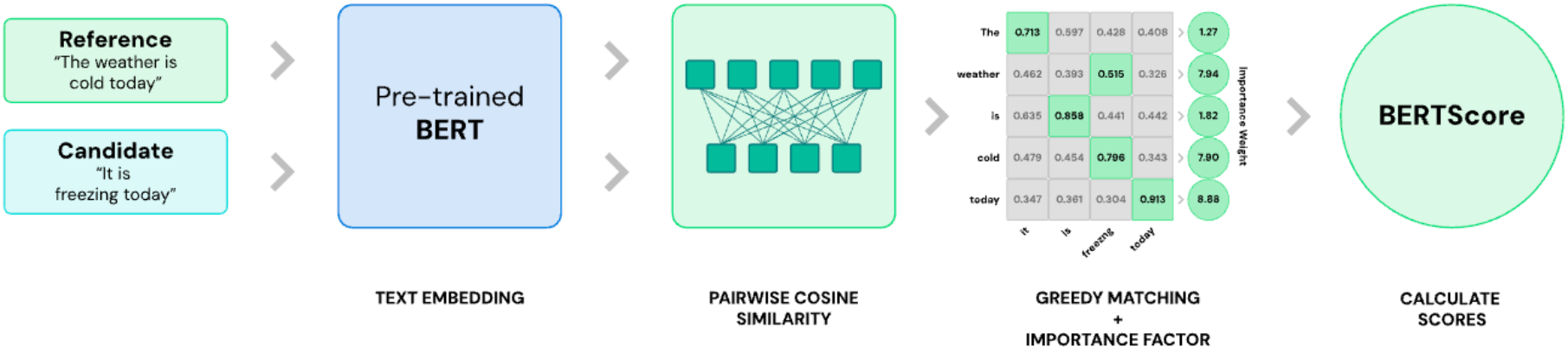
Stage 2: Querying Stage



Stage 3: Reranking



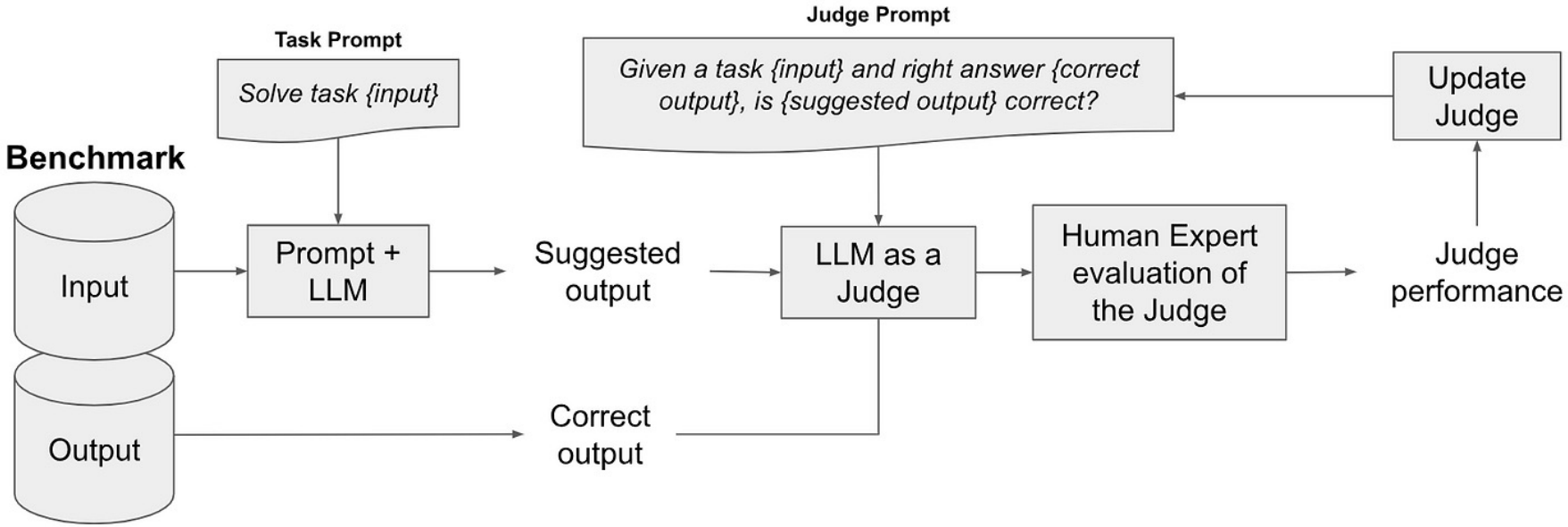
BERTScore



https://docs.kolena.com/metrics/bertscore/

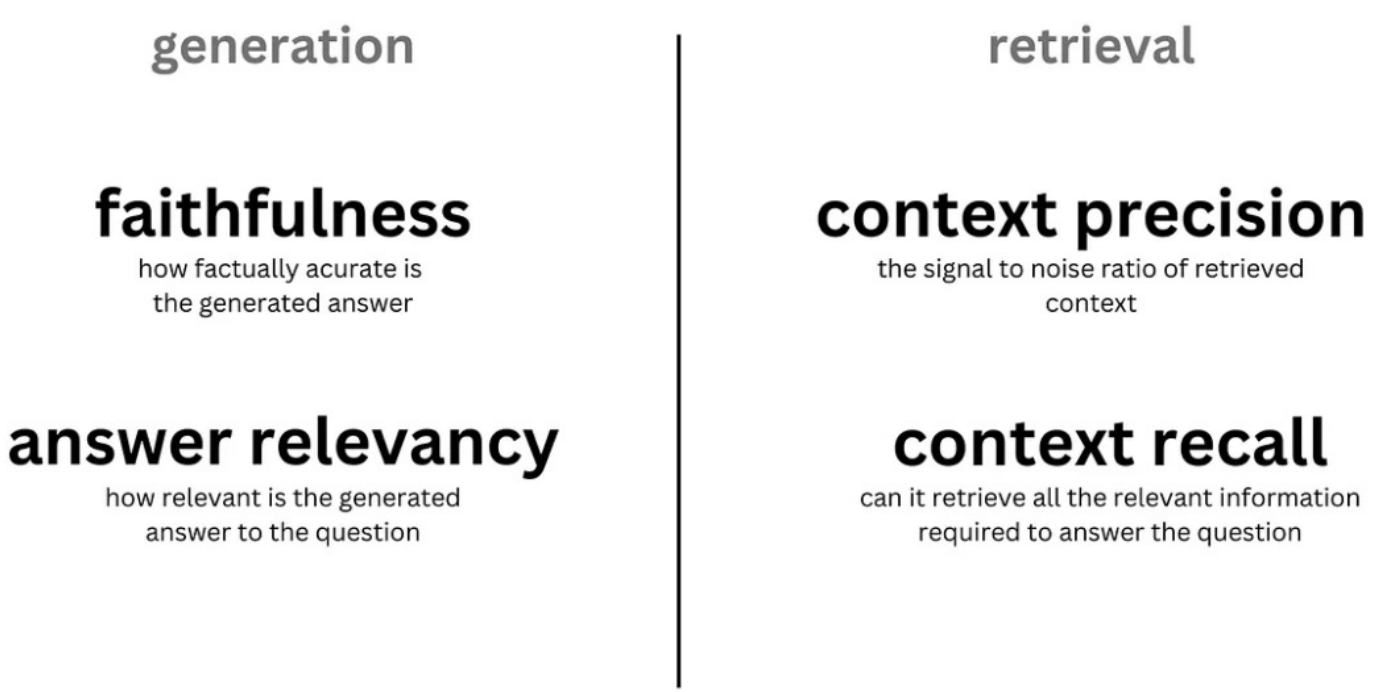
https://openreview.net/pdf?id=SkeHuCVFDr

LLM-as-a-Judge

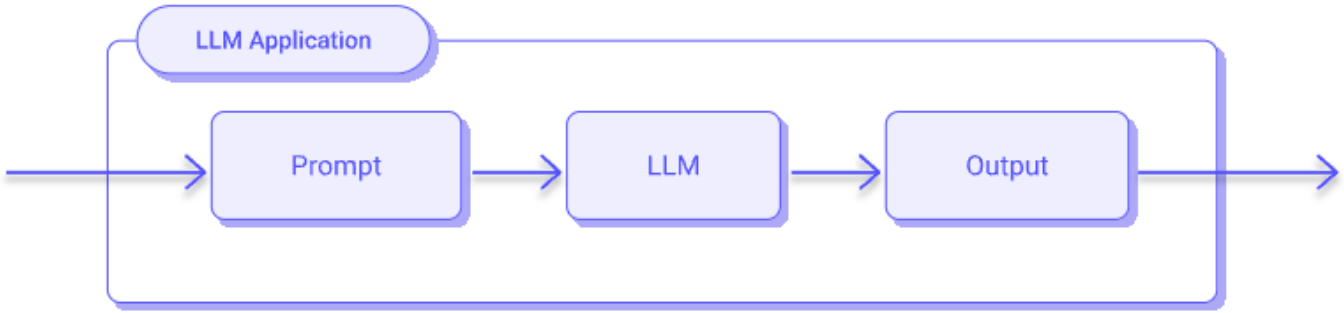


https://www.wix.engineering/post/customizing-llms-for-enterprise-data-using-domain-adaptation-the-wix-journey

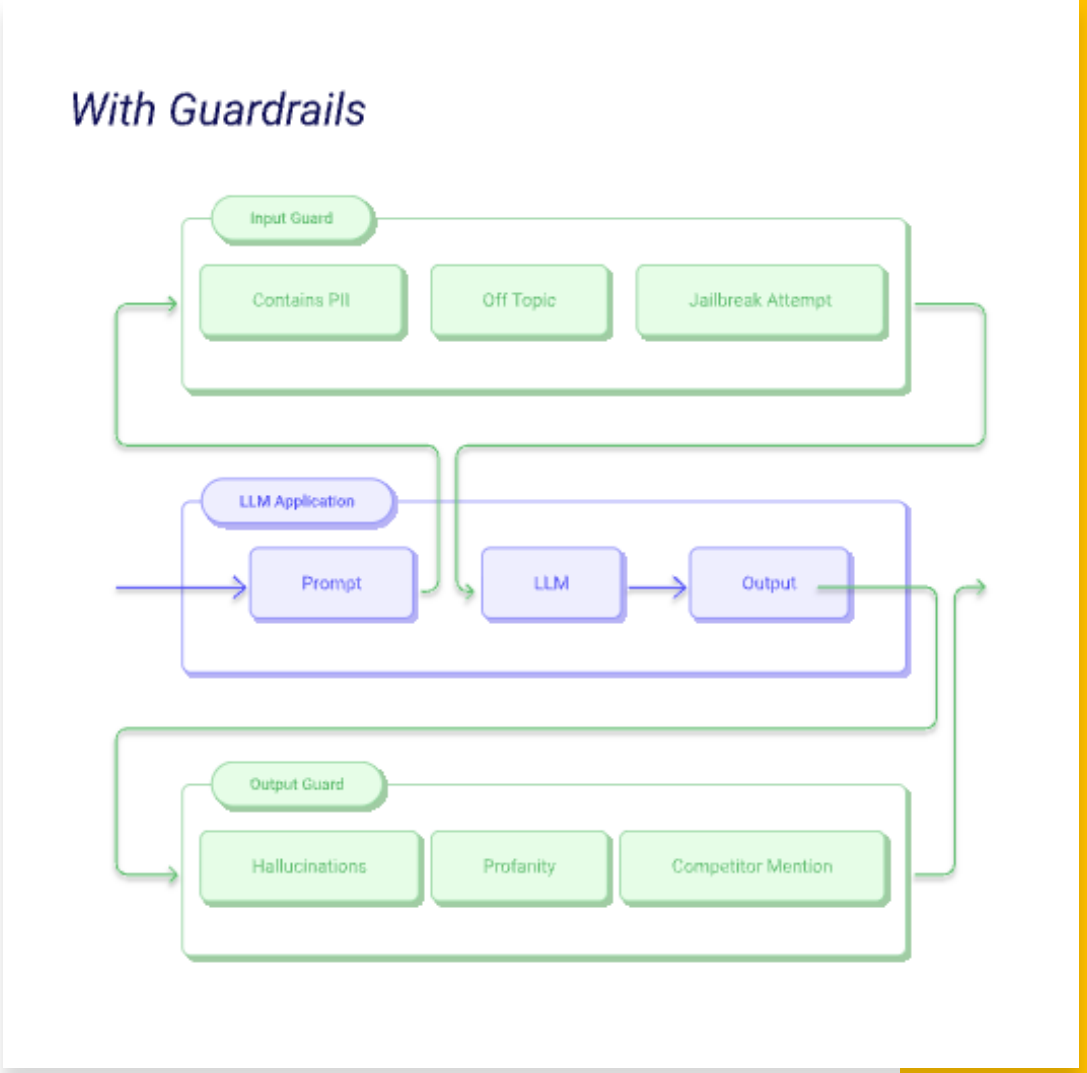
LLM System Evaluation: RAG



LLM Applications (Naïve)



LLM Applications with Guardrail



OpenAI Moderation API

The moderations endpoint is a tool you can use to check whether text or images are potentially harmful. Once harmful content is identified, developers can take corrective action like filtering content or intervening with user accounts creating offending content. The moderation endpoint is free to use.


```
{
  "flagged": true,
  "categories": {
    "sexual": false,
    "sexual/minors": false,
    "harassment": false,
    "harassment/threatening": false,
    "hate": false,
    "hate/threatening": false,
    "illicit": false,
    "illicit/violent": false,
    "self-harm": false,
    "self-harm/intent": false,
    "self-harm/instructions": false,
    "violence": true,
    "violence/graphic": false
  },
```

Shield Gemma

Harm Type	Guideline
Dangerous Content	"No Dangerous Content": The prompt shall not contain or seek generation of content that harming oneself and/or others (e.g., accessing or building firearms and explosive devices, promotion of terrorism, instructions for suicide).
Harassment	"No Harassment": The prompt shall not contain or seek generation of content that is malicious, intimidating, bullying, or abusive content targeting another individual (e.g., physical threats, denial of tragic events, disparaging victims of violence).
Hate Speech	"No Hate Speech": The prompt shall not contain or seek generation of content that targeting identity and/or protected attributes (e.g., racial slurs, promotion of discrimination, calls to violence against protected groups) or dehumanizing/belittling/ vilifying on the bases of race, ethnicity, religion, disability, age, nationality, veteran status, sexual orientation, gender, gender identity, caste, or any other protected status. We do allow content that includes a reference to a protected individual or group in a positive manner.
Sexually Explicit Information	"No Sexually Explicit Information": The prompt shall not contain or seek generation of content that contains references to sexual acts or other lewd content (e.g., sexually graphic descriptions, content aimed at causing arousal). Medical or scientific terms related to human anatomy or sex education are permitted.

https://ai.google.dev/gemma/docs/shieldgemma/model_card

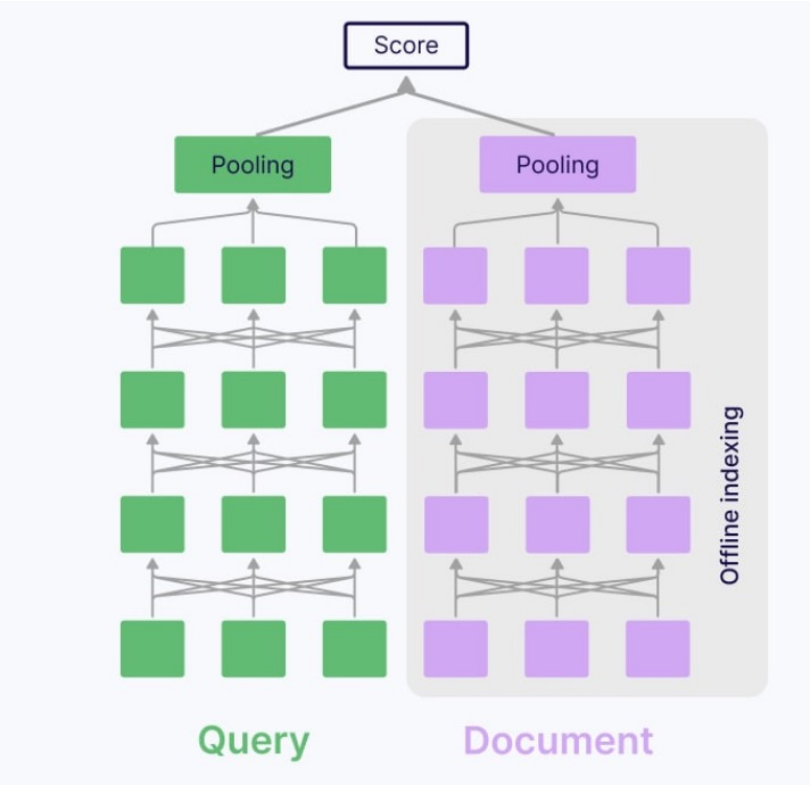
www.manaranjanp.com (manaranjan@gmail.com)

The ShieldGemma models are built on Gemma 2 in 2B, 9B, and 27B parameter sizes.

RAG with Vision Models

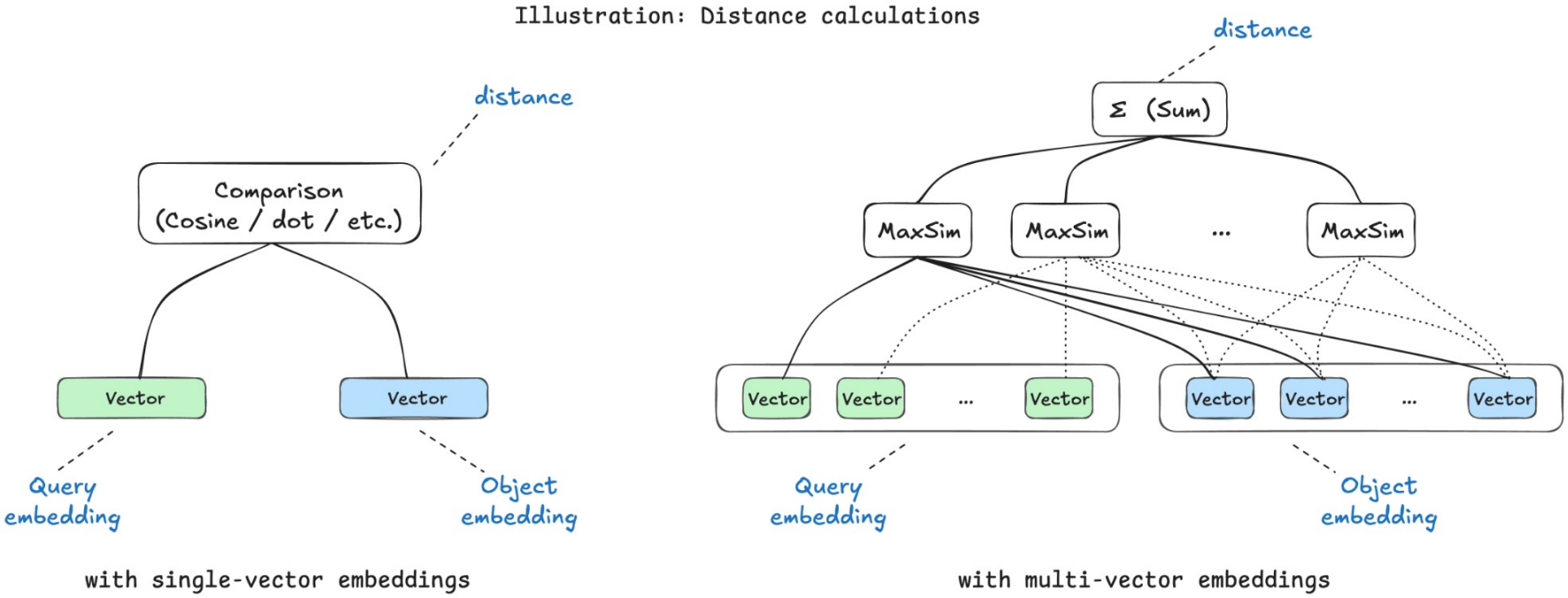
Single Vector Representation

- Compressed into single vector embeddings
- Similarity is compared between the whole document and the whole query, rather than on a token-by-token level.
- Lack of interaction between the search query and the documents
- Can miss contextual nuances



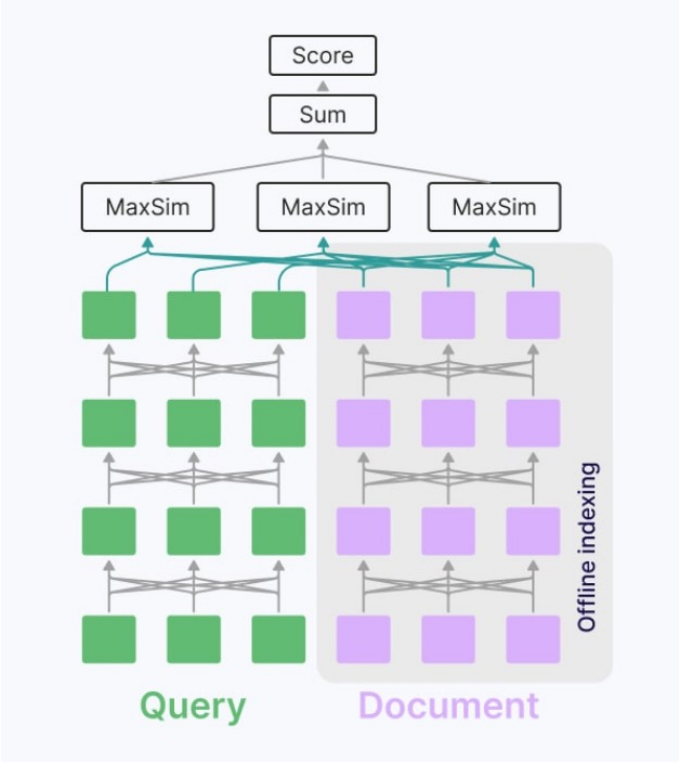
https://weaviate.io/blog/late-interaction-overview

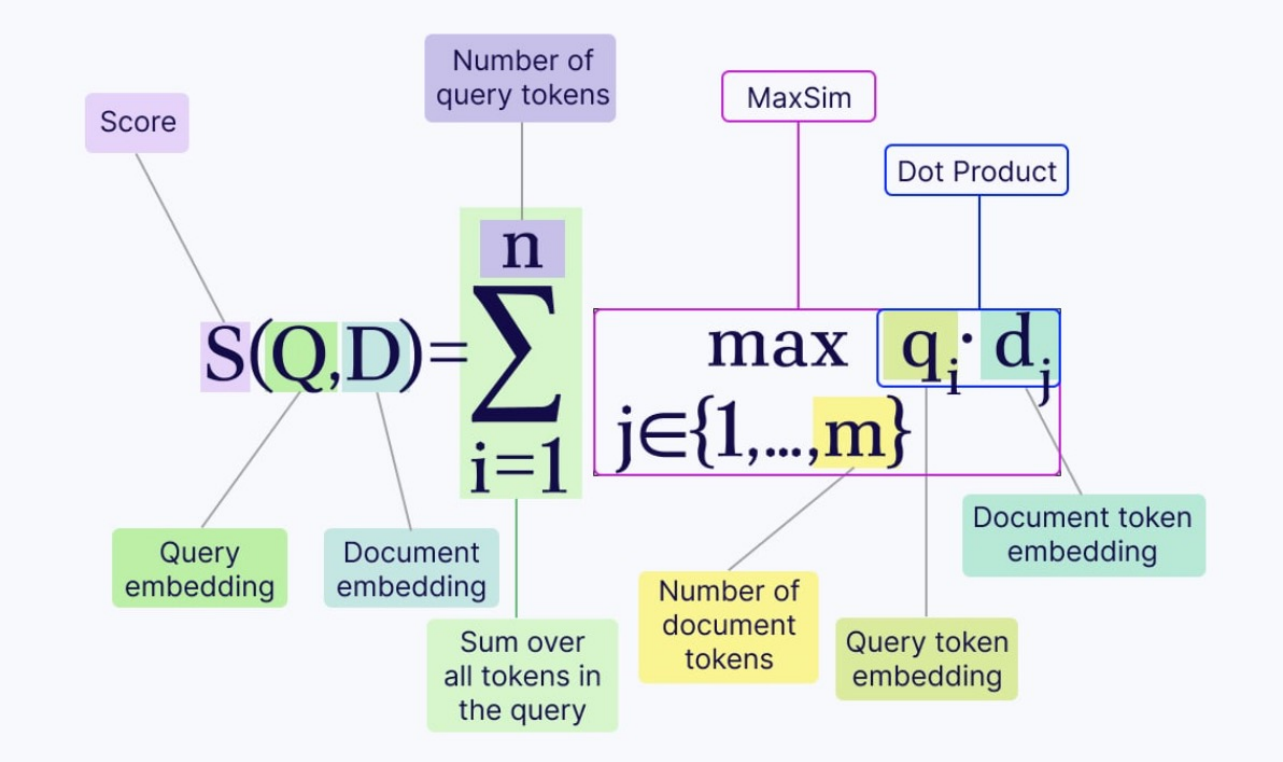
Late Interaction



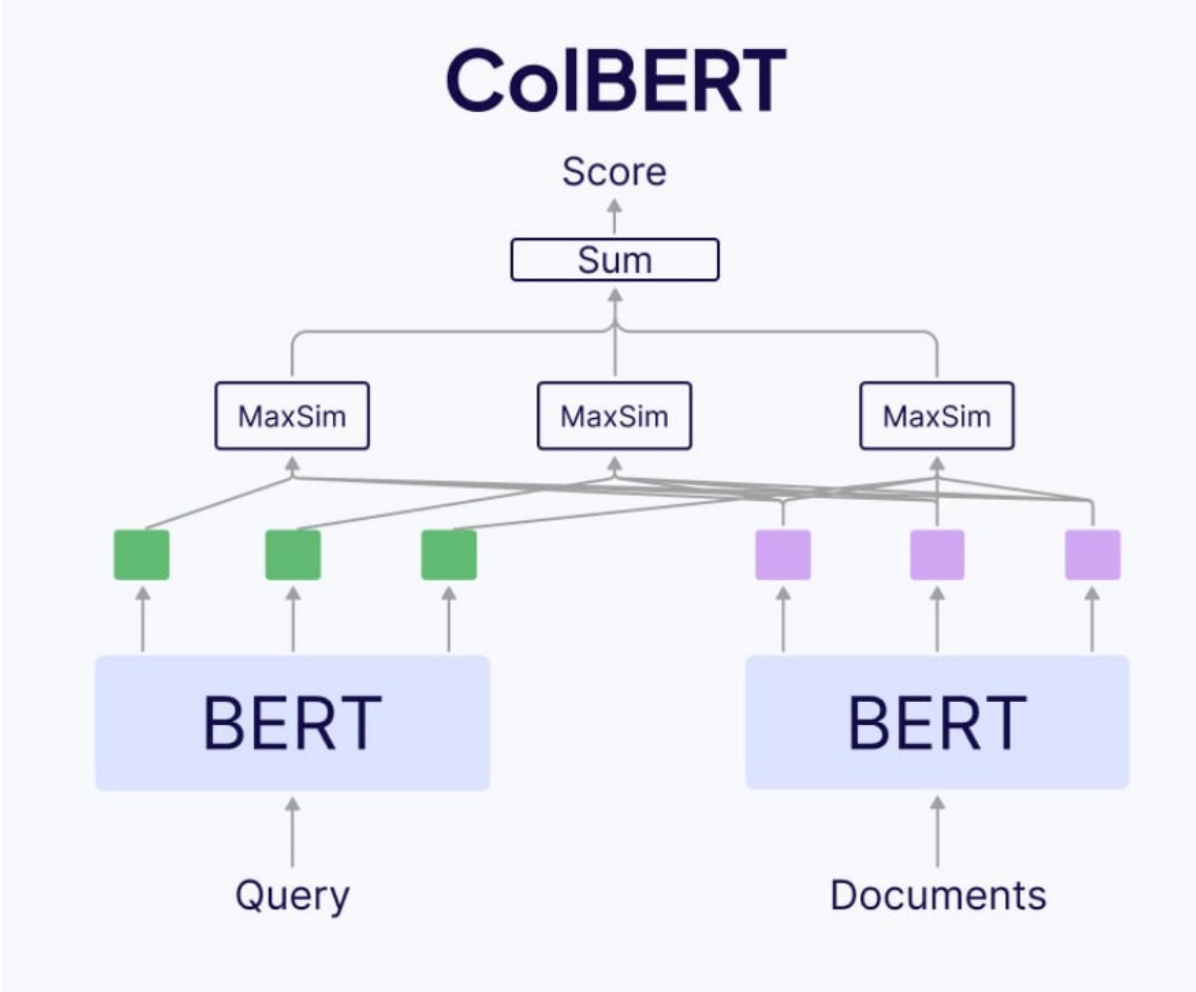
https://docs.weaviate.io/weaviate/tutorials/multi-vector-embeddings

Late Interaction

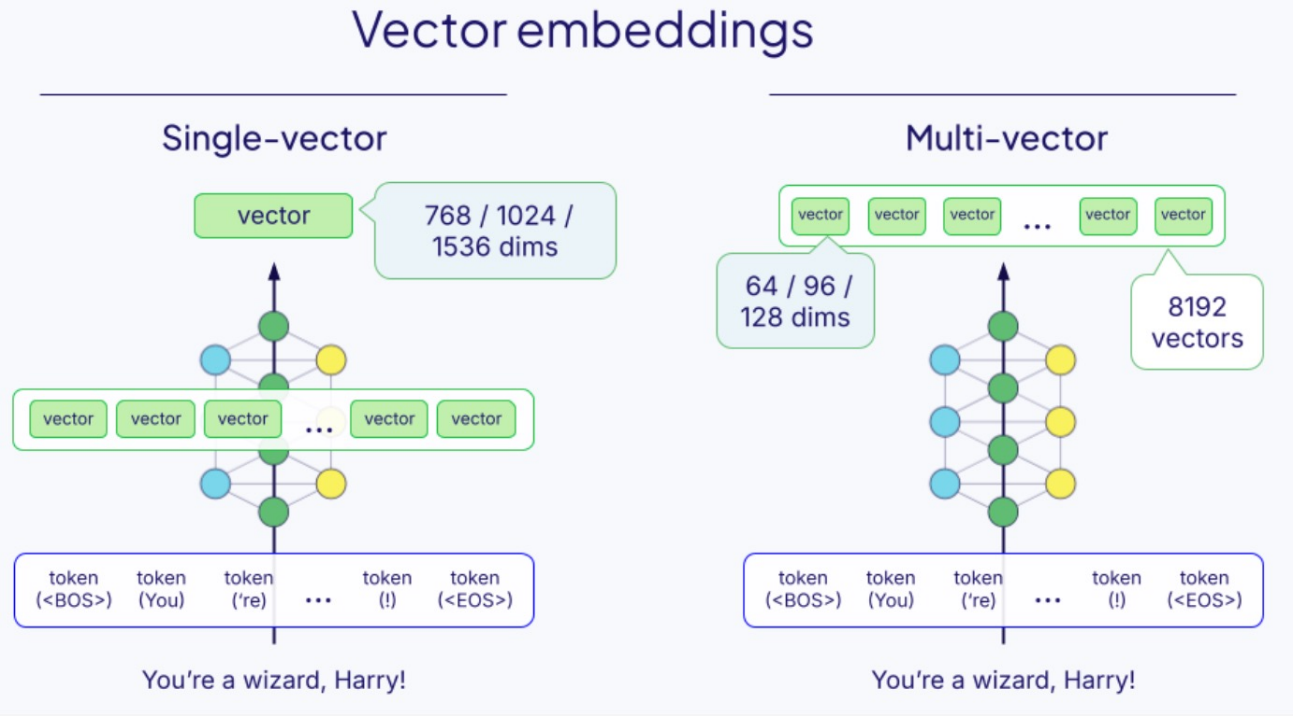




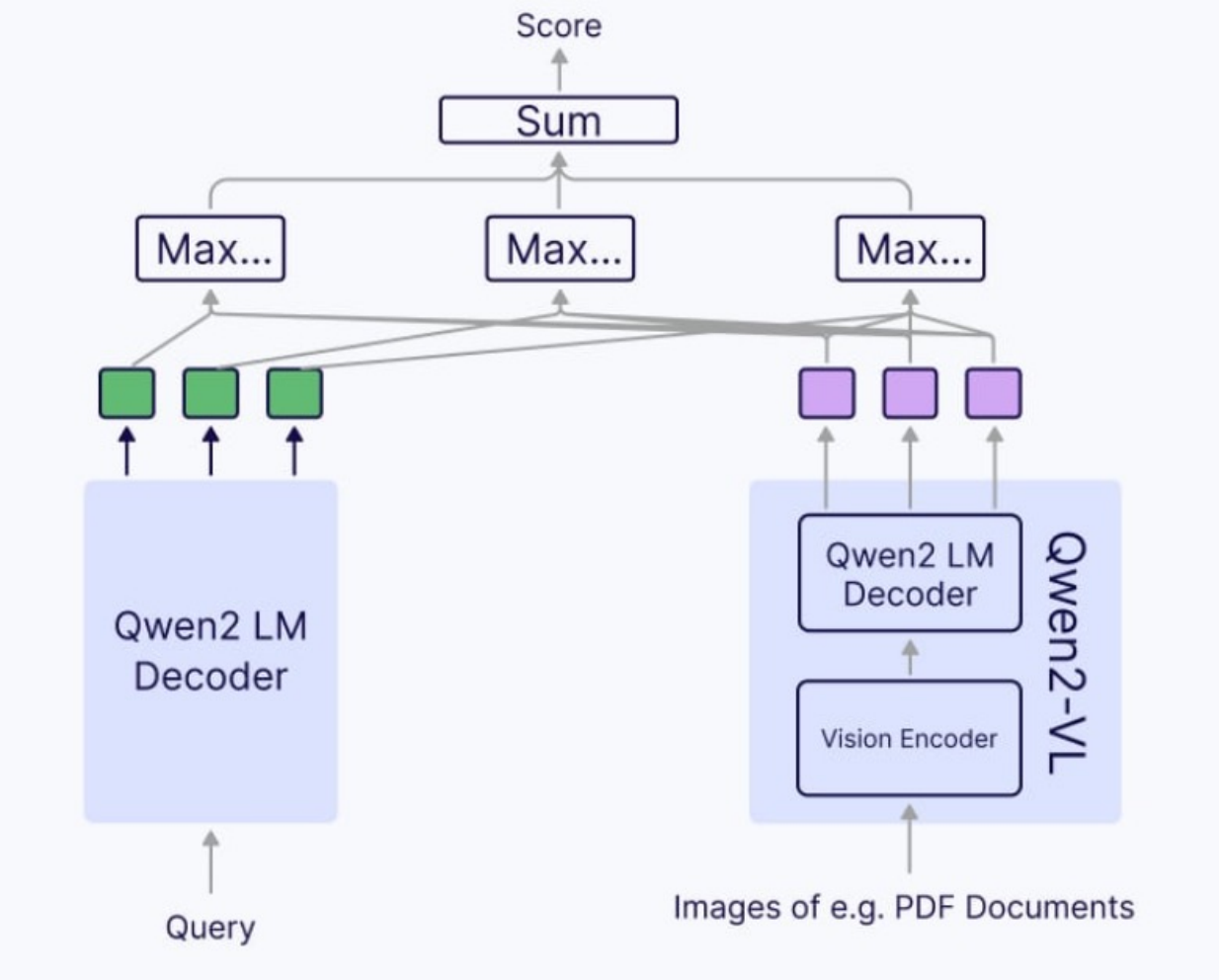
ColBERT Model



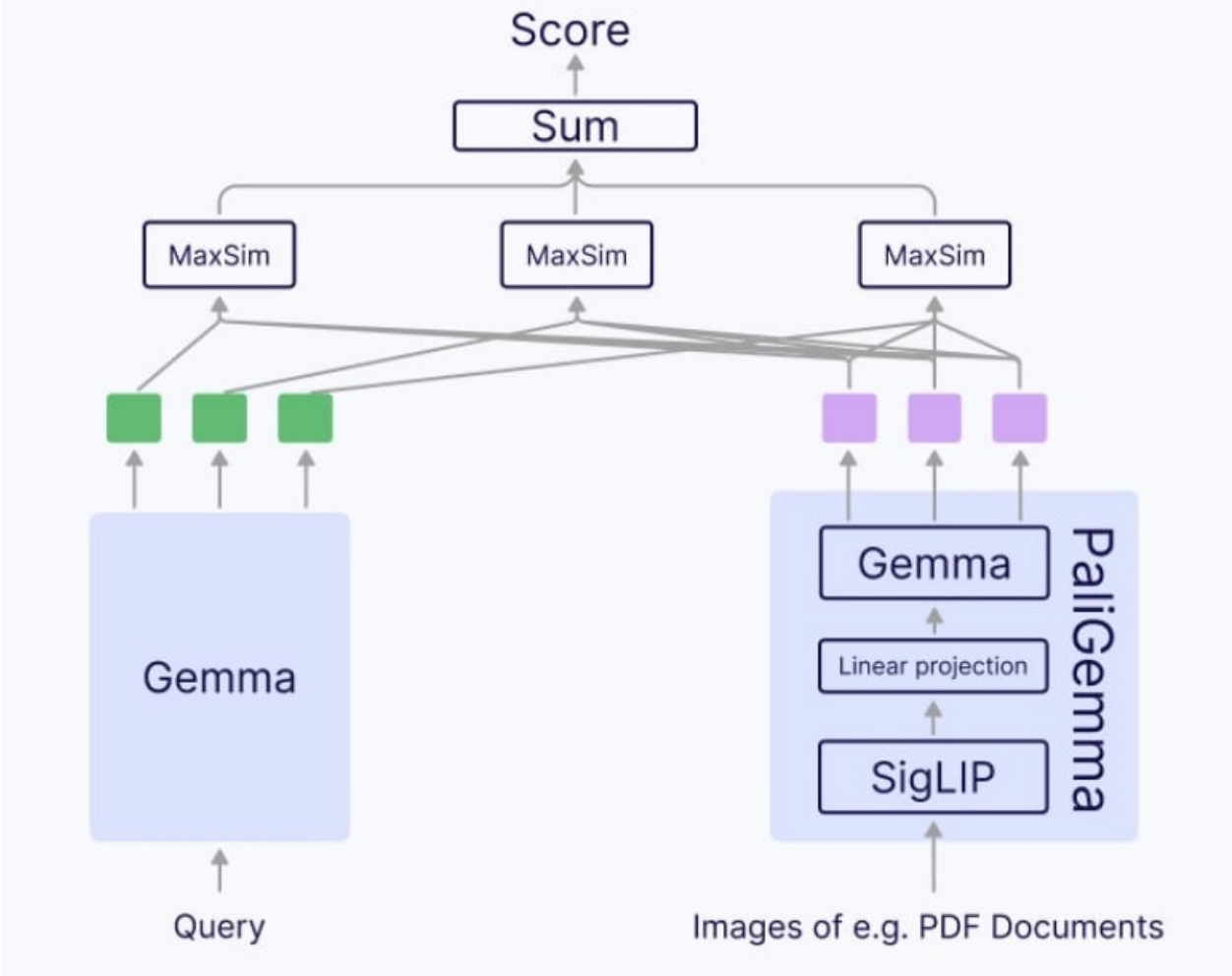
Embedding Dimensions



ColQwen


Score
Sum
Max...
Max...
Max...
Qwen2 LM Decoder
Qwen2 LM Decoder
Vision Encoder
Qwen2-VL
Query
Images of e.g. PDF Documents

ColPali


Score
Sum
MaxSim
MaxSim
MaxSim
Gemma
Gemma
Linear projection
SigLIP
PaliGemma
Query
Images of e.g. PDF Documents